ARCHITECTURE BLUEPRINT · MENU LINKS SYSTEM

Necoyoad

v7

The menu and menu_link system: admin CRUD, storefront rendering via the links widget, and tree composition

scope: menu/menu_link tables, admin controller (495 lines), admin model (274 lines), storefront model, links widget, 10+ link templates, EAV property + description tables
findings: recursive tree (N+1 queries), 3 submenu types, var_dump in production, every cross-cutting pattern
listings: 8 PHP · pages: 11

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — Menu Links in Admin and Apps (v7)

The menu and menu_link system: admin CRUD,
storefront rendering via the links widget, and tree composition

Abstract

This is the seventh volume of the Necoyoad architecture blueprint. v7 covers the menu and menu_link system: how the admin creates named menus scoped to stores and positions, how menu_link rows form a tree via `parent_id`, how the polymorphic `description` table provides localised link labels, how the EAV `property` table adds per-link metadata (CSS class, submenu type, icon, page ID), and how the storefront's "links" widget module renders the tree recursively via `ModelContentMenu::getAllItems()`.

The menu system is a microcosm of the Necoyoad architecture: it uses the AdminController declarative CRUD base (v6), the polymorphic `description` table (v5), the EAV `property` table (v3, v5), the `object_to_store` junction (v5), the per-store settings model (v5), and the widget rendering pipeline (v3). v7 traces the full lifecycle: admin creates a menu, adds links (with tree structure, localised labels, per-link CSS/icon/submenu-type), assigns the menu to a store and position, and the storefront renders it via the "links" widget which calls `getLinks()` recursively.

v7 documents this with 8 PHP code listings, 3 TikZ diagrams, and the full menu/-menu_link schema. The stance is unchanged: **descriptive, not prescriptive**.

Keywords: Necoyoad, OpenCart, PHP 8.1, menu, menu_link, navigation, links widget, tree, EAV property, polymorphic description, AdminController, object_to_store.

Contents

1 Introduction

1.1 Seven Volumes

v1–v6 covered the whole platform, source verification, the rendering pipeline, the marketing subsystem, multi-store/multi-language, and the admin back-office. v7 covers the menu links system — a focused subsystem that exercises every cross-cutting pattern.

1.2 Scope

v7 covers:

- The `menu` table — named menus scoped to stores and positions.
- The `menu_link` table — tree-structured links via `parent_id`.
- The admin menu controller (`ControllerContentMenu`) — an AdminController subclass with declarative CRUD.
- The admin menu model (`ModelContentMenu`) — with `on('save')` and `on('delete')` hooks for managing items.
- The storefront menu model — `getLinks()` and `getAllItems()` with caching.
- The storefront "links" widget module (`ControllerModuleLinks`) — renders a menu tree recursively.
- The EAV `property` table usage for per-link metadata (CSS class, submenu type, icon, page ID).
- The polymorphic `description` table for localised link labels.

2 The Menu Schema

2.1 The `menu` Table

```
CREATE TABLE `menu` (
  `menu_id`      int(11) NOT NULL,
  `store_id`     int(11) NOT NULL,
  `name`         varchar(100) NOT NULL,
  `position`     varchar(100) NOT NULL,    -- e.g. 'header', 'footer', 'sidebar'
  `sort_order`   int(11) NOT NULL,
  `route`        varchar(150) NOT NULL,    -- default route for this menu
  `status`       int(1) NOT NULL,
  `default`      int(1) NOT NULL,          -- is this the default menu for this
      position?
  `date_added`   datetime NOT NULL,
  `date_modified` datetime NOT NULL
);
```

Listing 1: The `menu` table schema.

A menu is a named collection of links, scoped to a store (`store_id`) and a position (`position`). The `position` field determines where on the page the menu appears (e.g. `header`, `footer`, `sidebar`). The `default` flag marks one menu per position as the default.

2.2 The `menu_link` Table

```
CREATE TABLE `menu_link` (
  `menu_link_id` int(11) NOT NULL,
  `menu_id`      int(11) NOT NULL,
  `parent_id`    int(11) NOT NULL,       -- tree via parent_id (0 = root)
  `link`         varchar(250) NOT NULL,  -- the URL
  `tag`          varchar(250) NOT NULL,  -- the visible label
  `sort_order`   int(11) NOT NULL
);
```

Listing 2: The `menu_link` table schema.

Menu links form a tree via `parent_id` (0 = root). Each link has a `link` (the URL) and a `tag` (the visible label). The `sort_order` field controls the display order within the same parent.

2.3 The Polymorphic Description and Property Extensions

Menu links use two polymorphic tables for additional data:

- `description` — localised link labels. The model's `$description_object_type = 'menu_link'`, so `description` rows with `object_type = 'menu_link'` and `object_id = <menu_link_id>` provide per-language `title`, `description` (rich HTML for submenu content), and SEO fields.
- `property` — per-link metadata. The model stores: `class_css` (a CSS class for the link), `submenu_type` (`none`, `page_id`, or `html_content`), `icon` (an image path), `page_id` (if `submenu_type = 'page_id'`, the page to embed as the submenu content).

3 The Admin Menu Controller

`app/admin/controller/content/menu.php` (495 lines) extends `ControllerAdmin` (v6) with declarative configuration:

```
class ControllerContentMenu extends ControllerAdmin {
    protected string $object_type       = 'menu';
    protected string $model_name        = 'modelMenu';
    protected string $model_route       = 'content/menu';
    protected string $model_object_type = 'menu';
    protected string $controller_name   = 'menu';
    protected string $controller_route  = 'content/menu';

    protected array $form_vars = [
        'menu_id'   => ['name' => 'menu_id', 'type' => 'number'],
        'parent_id' => ['name' => 'parent_id', 'type' => 'number'],
        'name'      => ['name' => 'name', 'type' => 'string'],
        'sort_order'=> ['name' => 'sort_order', 'type' => 'number'],
        'default'   => ['name' => 'default', 'type' => 'boolean'],
        'stores'    => ['name' => 'stores', 'type' => 'array'],
    ];

    protected array $public_methods = ['insert', 'update', 'copy',
        'delete', 'activate', 'grid'];

    public function init() {
        parent::init();
        $this->addFilter("grid:data", function ($data) {
            // Configure grid columns, batch actions, etc.
            return $data;
        });
    }
}
```

Listing 3: `ControllerContentMenu` — declarative configuration.

Because it extends `AdminController`, it inherits `index()`, `insert()`, `update()`, `delete()`, `activate()`, `grid()`, `getList()`, `getForm()`, `validateForm()`, and `validateDelete()` — all driven by the declarative configuration. The controller only adds a `grid:data` filter to customise the grid columns.

4 The Admin Menu Model

`app/admin/model/content/menu.php` (274 lines) extends `Model` with declarative table/field configuration and `on('save')`/`on('delete')` hooks:

```
class ModelContentMenu extends Model {
    protected string $table        = "menu";
    protected string $pkey         = "menu_id";
    protected string $object_type  = "menu";
    protected string $description_object_type = "menu_link";

    protected array $fields = [
        "store_id"      => ["type" => "integer", "default" => 0],
        "name"          => ["type" => "string"],
        "status"        => ["type" => "boolean", "default" => 1],
        "date_added"    => ["type" => "sql", "default" => "NOW()"],
        "date_modified" => ["type" => "sql", "default" => "NOW()"],
    ];

    protected array $relations = ["stores"];

    public function init() {
        $this->on("save", function($args) {
            $data = $args[0];
            if ($data["action"] == "update") $this->deleteItems($data['id']);
            if ($data['id']) $this->setItems($data['id'],
                $data['data']['link']);
        });

        $this->on("delete", function ($args) {
            $data = $args[0];
            $this->deleteItems($data['id']);
        });
    }
}
```

Listing 4: `ModelContentMenu` — declarative model with save/delete hooks.

The `on('save')` hook fires after the menu is saved: if updating, it first deletes all existing `menu_link` rows for this menu, then re-inserts them from `$data['data']['link']` (the form's link data). The `on('delete')` hook deletes all `menu_link` rows when a menu is deleted.

The `setItems()` method inserts `menu_link` rows with their tree structure (`parent_id`), URL (`link`), label (`tag`), and `sort_order`. It also stores per-link properties in the EAV `property` table.

5 The Storefront Menu Model

`app/shop/model/content/menu.php` (simplified) provides the storefront's read-only access to menu links:

```php
public function getAllItems($data = null) {
    $cache_prefix = "menu_links";
    $cachedId = $cache_prefix . (int)STORE_ID . "_" .
        serialize($data) .
        $this->config->get('config_language_id') . "." .
        $this->request->getQuery('hl') . "." .
        $this->request->getQuery('cc') . "." .
        $this->config->get('config_currency') . "." .
        (int)$this->config->get('config_store_id');

    $cached = $this->cache->get($cachedId, $cache_prefix);
    if (!$cached || (bool)$this->user->getId()) {
        $sql = "SELECT * FROM " . DB_PREFIX . "menu_link ml ";
        // ... WHERE menu_id = $data['menu_id'] AND parent_id =
            $data['parent_id']
        // ... ORDER BY sort_order ASC
        // ... also loads property table for class_css, submenu_type, icon,
            page_id
        // ... also loads description table for localised tag
        $this->cache->set($cachedId, $result, $cache_prefix);
    }
    return $cached;
}

public function getLinks($menu_id, $parent_id) {
    return $this->getAllItems([
        'menu_id'   => $menu_id,
        'parent_id' => $parent_id,
    ]);
}
```

Listing 5: Storefront `ModelContentMenu::getAllItems()` — cached menu link retrieval.

The cache key includes `STORE_ID`, `language_id`, `hl`, `cc`, and `currency` — so the cached menu links are store- and language-specific. The cache is bypassed for admins.

6 The Storefront "Links" Widget Module

The storefront renders menus via the "links" widget module (`app/shop/controller/module/links.php`). This is a standard widget module (covered in v3) that extends `ControllerModuleModuleController`.

```php
class ControllerModuleLinks extends ControllerModuleModuleController {
    protected string $moduleName = 'links';

    public function init() {
        $this->addFilter("module:settings", function ($data) {
            $settings = $data['settings'];
            $widget   = $data['widget'];
            $render   = $data['render'];

            // Render the menu tree as HTML
            $this->data['links'] = isset($settings['menu_id']) &&
                (int)$settings['menu_id'] > 0
                ? $this->drawLinksGroup($this->getLinks($settings['menu_id']))
                : '';

            return [
                'widget'   => $widget,
                'render'   => $render,
                'settings' => $settings,
```

```
            ];
        });
    }

    protected function getLinks($menu_id = 0, $parent_id = 0) {
        $this->load->model('content/menu');
        $this->load->model('content/page');

        $return = [];
        $results = $this->modelMenu->getAllItems([
            'menu_id'   => $menu_id,
            'parent_id' => $parent_id,
        ]);

        if ($results) {
            foreach ($results as $k => $result) {
                $return[$k] = $result;

                // Load per-link EAV properties
                $result['class_css']   =
                    $this->modelMenu->getProperty($result['menu_link_id'],
                    'menu_link', 'class_css');
                $result['submenu_type'] =
                    $this->modelMenu->getProperty($result['menu_link_id'],
                    'menu_link', 'submenu_type');
                $result['icon']        =
                    $this->modelMenu->getProperty($result['menu_link_id'],
                    'menu_link', 'icon');

                if ($result['submenu_type'] === 'page_id') {
                    // Embed a CMS page as the submenu content
                    $result['page_id'] =
                        $this->modelMenu->getProperty($result['menu_link_id'],
                        'menu_link', 'page_id');
                    if (!empty($result['page_id'])) {
                        $pageController =
                            $this->load->controller('content/page');
                        $return[$k]['description'] =
                            html_entity_decode($pageController->embed($result['page_id']));
                    }
                } elseif ($result['submenu_type'] === 'html_content') {
                    // Load localised HTML content from the description table
                    $descriptions =
                        $this->modelMenu->getDescriptions($result['menu_link_id']);
                    $return[$k]['description'] = html_entity_decode(
                        $descriptions[$this->config->get('config_language_id')]['descriptio
                            ?? ""
                    );
                } else {
                    // Recurse into children
                    $return[$k]['children'] = $this->getLinks($menu_id,
                        $result['menu_link_id']);
                }
            }
        }
        return $return;
    }
}
```

Listing 6: `ControllerModuleLinks` — the links widget (abridged).

6.1 The Recursive Tree Rendering

The `getLinks()` method is recursive: for each link, it calls itself with `parent_id = menu_link_id` to fetch the link's children. This builds a tree structure:

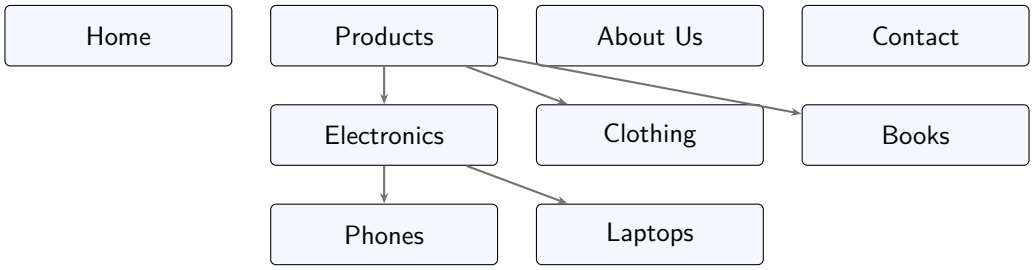

Figure 1: A menu link tree. The `getLinks()` method recurses through `parent_id` to build this structure. Each level is a separate `SELECT * FROM menu_link WHERE parent_id = ...` query (unless cached).

6.2 The Three Submenu Types

Each menu link can have one of three submenu types (stored in the `property` EAV table under `group='menu_link'`, `key='submenu_type'`):

Table 1: The three submenu types.

Type	Data source	Rendering
`none` (default)	Children from `menu_link`	Recurse into `getLinks()` for child links.
`page_id`	`property` key `page_id`	Embed a CMS page's content as the submenu (calls `content/page->embed()`).
`html_content`	`description` table	Load localised HTML from the polymorphic `description` table for this `menu_link_id`.

6.3 The `drawLinksGroup()` Method

The `drawLinksGroup()` method takes the tree array from `getLinks()` and renders it as nested `<ul>/<li>` HTML. The exact HTML structure depends on the widget's `settings['view']` (which selects the template: `links_main_menu.tpl`, `links_default.tpl`, `links_vertical.tpl`, etc.).

6.4 The Link Templates

The theme ships 10+ link templates:

- `links_main_menu.tpl` — the main navigation bar.
- `links_default.tpl` — a simple `<ul>` list.
- `links_vertical.tpl` — a vertical sidebar menu.
- `links_overheader.tpl` — links above the header.
- `links_01.tpl` through `links_07.tpl` — numbered variants for different layout positions.
- `links_marketo.tpl` — a marketo-style dropdown.
- `link_button.tpl` / `link_button_default.tpl` — a single button-style link (used by the separate `link_button` module).

7 The Menu Lifecycle Trace

7.1 Admin Creates a Menu

1. Admin navigates to `?r=content/menu/insert`.
2. `ControllerContentMenu::insert()` calls `AdminController::upsert()`.
3. `upsert()` validates the form, processes POST data, calls `modelMenu->add($data)`.
4. `ModelContentMenu::add()` inserts the `menu` row.
5. The `on('save')` hook fires: calls `setItems($menu_id, $data['link'])`, which inserts `menu_link` rows (with tree structure via `parent_id`) and per-link `property` EAV rows (class_css, submenu_type, icon, page_id).
6. The `on('save')` hook also writes `object_to_store` rows (via the `relations = ["stores"]` declaration) to scope the menu to stores.
7. Admin activity is logged in `user_activity`.
8. Redirect to `content/menu` (list view).

7.2 Storefront Renders the Menu

1. A page loads with a "links" widget configured with `menu_id = 5`.
2. `ControllerModuleLinks::init()` registers the `module:settings` filter.
3. `ControllerModuleModuleController::index()` calls the filter, which calls `drawLinksGroup(getLinks(5))`.
4. `getLinks(5, 0)` calls `modelMenu->getAllItems(['menu_id'=>5, 'parent_id'=>0])`.
5. `getAllItems()` checks the cache (keyed by `STORE_ID + language_id + ...`). On cache miss, queries `SELECT * FROM menu_link WHERE menu_id = 5 AND parent_id = 0 ORDER BY sort_order`, then loads per-link properties from `property` and localised labels from `description`.
6. For each root link, `getLinks(5, <menu_link_id>)` is called recursively to fetch children.
7. For each link with `submenu_type = 'page_id'`, `content/page->embed($page_id)` is called to embed the page content.
8. For each link with `submenu_type = 'html_content'`, `modelMenu->getDescriptions($menu_link_id)` is called to load the localised HTML.
9. `drawLinksGroup()` renders the tree as nested `<ul>/<li>` HTML using the selected template.
10. The HTML is returned via the `module:settings` filter and stored in `$this->data['links']`.
11. The widget template (`links_main_menu.tpl`) emits `<?php echo $links; ?>`.
12. `Controller::fetch()` substitutes the `{%<widget_name>%}` placeholder with the rendered HTML (v3's composite-view pattern).

8 Cross-Cutting Findings

8.1 The Menu System Exercises Every Cross-Cutting Pattern

The menu system is a microcosm of the Necoyoad architecture:

- **AdminController declarative CRUD** (v6): the menu controller is 495 lines, but most of that is the `grid:data` filter and form customisation. The actual CRUD is inherited from AdminController.
- **Polymorphic `description` table** (v5): menu links use `object_type = 'menu_link'` for localised labels and HTML submenu content.
- **EAV `property` table** (v3, v5): per-link metadata (CSS class, submenu type, icon, page ID) stored as key-value pairs under `group = 'menu_link'`.
- **`object_to_store` junction** (v5): menus are scoped to stores via the `relations = ["stores"]` declaration.
- **Widget rendering pipeline** (v3): the links widget extends `ControllerModuleModuleController` and uses the `module:settings` filter to inject the rendered menu tree.
- **Multi-store/multi-language** (v5): the storefront cache key includes `STORE_ID` and `config_language_id`, so each (store, language) pair has its own cached menu tree.

8.2 The Recursive Tree Rendering is N+1 Query Prone

N+1 query pattern in tree rendering

The `getLinks()` method recurses through the tree by calling `getAllItems()` for each parent. Each call is a separate `SELECT * FROM menu_link WHERE parent_id = ...` query, plus per-link `property` and `description` lookups. For a 3-level menu with 5 root items, 3 children each, and 2 grandchildren each, this is $1 + 5 + 15 = 21$ queries for the tree, plus $21 \times 2 = 42$ property/description queries — 63 queries total.
The cache (keyed by store + language + data) absorbs this on subsequent requests, but the first request (or any request after a cache clear) pays the full N+1 cost.

8.3 The `var_dump` Left in Production Code

`var_dump()` in the links widget

The `ControllerModuleLinks::getLinks()` method contains a `var_dump($descriptions)` call (line 82) that fires when `submenu_type === 'html_content'`. This is debug code left in production. Every menu link with an HTML-content submenu type will dump the descriptions array to the browser before the menu HTML.

8.4 The Menu Link Label is Both in `menu_link.tag` and `description.title`

The `menu_link` table has a `tag` column (the visible label), but the polymorphic `description` table also has a `title` column for localised labels. The storefront model loads both. If a localised description exists, it presumably overrides the `tag`; if not, the `tag` is used. This dual storage (non-localised `tag` + localised `description.title`) is the same pattern used by products (`product.model` + `description.title`).

9 Closing Observations

9.1 The Menu System is a Clean Application of the Architecture

The menu system is one of the cleanest applications of the Necoyoad architecture. The admin controller is almost entirely declarative (via AdminController), the model uses the standard `on('save')/on('delete')` hook pattern, the storefront model uses the standard cache pattern, and the widget module uses the standard `module:settings` filter pattern. No special cases, no custom SQL (beyond the model's declarative fields), no custom rendering logic (beyond the recursive tree).

9.2 The Menu System is Where the Patterns Compose

The menu system is where all the cross-cutting patterns compose: AdminController (v6) for CRUD, polymorphic description (v5) for localisation, EAV property (v3, v5) for metadata, object_to_store (v5) for multi-store, widget rendering (v3) for display, and multi-language caching (v5) for performance. v7 is the volume that ties them all together in a single subsystem.

9.3 Final Note

v7 confirms that the Necoyoad menu system is a clean, recursive, multi-store, multi-language navigation system built on the platform's cross-cutting patterns. The AdminController declarative CRUD, the polymorphic description table, the EAV property table, and the widget rendering pipeline all work together to produce a menu system that is both flexible (tree structure, three submenu types, per-link CSS/icon) and consistent with the rest of the platform.

A Glossary

Table 2: Glossary of menu system terms.

Term	Meaning
Menu	A named collection of links, scoped to a store and position.
Menu link	A single link within a menu, with tree structure via `parent_id`.
Submenu type	The `property` EAV key that controls how a link's submenu is rendered: `none` (children), `page_id` (embedded CMS page), or `html_content` (localised HTML).
Links widget	The storefront module (`ControllerModuleLinks`) that renders a menu tree recursively.
`drawLinksGroup()`	The method that converts the tree array into nested `<ul>/<li>` HTML.
`getLinks()`	The recursive method that fetches menu links from the model and builds the tree array.